

Project Report

Enterprise Autonomous Market Intelligence & Competitive Strategy Platform

Prepared by: Muhammad Hassan Tariq

Internship Organization: EZITECH Software House

Case Study: AI-004 (Ezitech Engineering Framework)

Abstract

The **Enterprise Autonomous Market Intelligence & Competitive Strategy Platform** is an AI-powered decision support system developed as part of the **EZITECH AI-004 Industry Case Study**. The objective of this project is to automate the process of collecting, analyzing, and presenting market intelligence for strategic business decision-making.

Modern organizations operate in highly competitive environments where large volumes of information are generated every day through news articles, financial reports, product announcements, customer reviews, and other public sources. Manually monitoring these sources is time-consuming and often delays strategic decisions. This platform addresses that challenge by continuously collecting market information, processing it through artificial intelligence, and presenting actionable insights through an executive dashboard.

The developed system integrates multiple AI technologies, including Natural Language Processing (NLP), Knowledge Graphs, Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), Time-Series Forecasting, and Multi-Agent AI. Live data is collected from several public sources, processed using NLP techniques, stored in PostgreSQL and Neo4j, indexed using ChromaDB, and analyzed by specialized AI modules. These modules generate SWOT analyses, strategic recommendations, competitor insights, market forecasts, and scenario simulations.

The backend is implemented using FastAPI, while PostgreSQL stores raw market data, Neo4j manages the knowledge graph, Redis provides caching for improved performance, and Groq-powered language models perform advanced reasoning tasks. An Executive Dashboard provides business users with interactive visualizations and AI-generated recommendations to support informed decision-making.

Although the original case study targets enterprise-scale deployment, this implementation focuses on demonstrating the complete system architecture using a manageable dataset while preserving all major architectural components and AI workflows. The project successfully demonstrates how multiple AI technologies can be integrated into a single intelligent platform for enterprise market intelligence and competitive strategy analysis.

Table of Content

Abstract	2
Table of Content	3
1. Introduction	5
1.1 Purpose of This Document	5
1.2 Industry Background	5
1.3 Business Problem	5
1.4 Project Objective	6
2. Project Scope & Approach	6
2.1 Project Scope	6
2.2 Implemented Features	6
2.3 Non-Implemented Features and Justification	7
2.4 Future Scalability	8
3. System Architecture	8
3.1 Architectural Overview	8
3.1.1 Layer 1: Raw Data Layer	9
3.1.2 Layer 2: Processing Layer	9
3.1.3 Layer 3: Knowledge Layer	9
3.1.4 Layer 4: AI Reasoning Layer	10
3.1.5 Layer 5: Presentation Layer	11
3.2 System Architecture Diagram	12
3.3 Data Flow	12
4. Technology Stack	13
4.1 Backend Technologies	13
4.2 AI Technologies	13
4.3 Database Technologies	14
4.3.1 PostgreSQL	14
4.3.2 Neo4j	14
4.3.3 Redis (Memurai)	14
4.4 Development Tools	14
4.4.1 Visual Studio Code	14
5. System Implementation	15
5.1 Multi-Source Data Intelligence	15
5.2 Live Data Ingestion Pipeline	15
5.3 PostgreSQL Raw Data Storage	19
5.4 NLP Pipeline	19
5.5 Knowledge Graph (Neo4j)	19
5.6 Semantic Search (RAG)	20
5.7 Forecasting Engine	21
5.8 SWOT Intelligence Generator	21
5.9 AI Strategy Advisor	24
5.10 Scenario Simulation Engine	25
5.11 AI Executive Copilot	27
5.12 Explainable AI	30
5.13 Executive Dashboard	31
5.14 Multi-Agent AI Architecture	31
5.15 Redis Caching	31
5.16 NetworkX Competitive Analysis	45
6. REST API Reference	46
6.1 API Overview	46
6.2 Endpoint Summary	47
6.3 Request and Response Examples	47
7. Database Design	47

7.1 PostgreSQL Schema	47
7.2 Neo4j Knowledge Graph Schema	49
7.3 ChromaDB Collection Structure	49
8. Deployment Overview	50
8.1 System Requirements	50
8.2 Software Components	50
8.3 Deployment Workflow	50
8.4 Environment Configuration Overview	51
9. Testing and Validation	51
9.1 Functional Testing	51
9.2 API Testing	51
9.3 Knowledge Graph Verification	51
9.4 Semantic Search Validation	52
9.5 Forecasting Validation	52
9.6 Dashboard Validation	52
10. Engineering Challenges and Solutions	52
10.1 Duplicate Knowledge Graph Nodes	52
10.2 Prophet Build Issues on Windows	53
10.3 Reddit API Limitations	53
10.4 JSON Parsing Issues	53
10.5 Executive Copilot Evidence Bug	54
10.6 NewsAPI Rate Limiting	54
10.7 NetworkX Centrality Improvements	54
11. Known Limitations and Future Work	55
11.1 Current Limitations	55
11.2 Future Enhancements	56
12. Conclusion	56
Appendix A. Project Architecture Diagram	57
Appendix B. API Endpoints Summary	57
Appendix C. Environment Variables	58

1. Introduction

1.1 Purpose of This Document

The purpose of this document is to provide a comprehensive technical description of the **Enterprise Autonomous Market Intelligence & Competitive Strategy Platform**. It explains the project's objectives, overall architecture, implementation methodology, AI components, database design, REST APIs, deployment overview, testing activities, and engineering decisions.

1.2 Industry Background

Organizations operating in global markets face increasing competition and rapidly changing business environments. New products, pricing strategies, acquisitions, partnerships, and technological innovations occur continuously across different industries. At the same time, valuable business information is distributed across numerous public sources, including news websites, financial reports, patent databases, customer reviews, and industry publications.

Traditional market research teams often require significant time to collect, organize, and analyze this information manually. As a result, important business opportunities may be missed, competitor activities may go unnoticed, and strategic decisions may be delayed.

Recent advances in Artificial Intelligence enable organizations to automate much of this process by combining machine learning, Natural Language Processing, Knowledge Graphs, Retrieval-Augmented Generation, and Large Language Models. These technologies allow enterprises to continuously monitor market activity, organize information intelligently, and generate evidence-based strategic recommendations.

1.3 Business Problem

The case study considers a multinational consumer electronics company operating across multiple countries, product categories, and brands while competing with hundreds of organizations worldwide. Every day, the company must analyze thousands of news articles, financial reports, customer reviews, and industry publications to understand market changes and competitor activities.

Manual analysis of such a large volume of information creates several challenges, including delayed competitor analysis, missed market opportunities, slow strategic planning, fragmented data sources, and limited forecasting capability. Executives require timely and reliable insights to make informed business decisions; however, traditional market research processes are often unable to deliver information quickly enough.

To address these challenges, an intelligent platform capable of automatically collecting, organizing, analyzing, and presenting market intelligence is required.

1.4 Project Objective

The primary objective of this project is to develop an AI-powered **Enterprise Autonomous Market Intelligence & Competitive Strategy Platform** capable of automating the market intelligence lifecycle.

The platform is designed to collect information from multiple public sources, process raw data using Natural Language Processing techniques, organize business relationships through a Neo4j Knowledge Graph, perform semantic retrieval using Retrieval-Augmented Generation (RAG), forecast competitor activity using time-series analysis, and generate explainable AI-based strategic recommendations.

In addition, the platform provides an Executive Dashboard that enables users to visualize market trends, competitor activities, business risks, innovation indicators, and AI-generated strategic insights through an intuitive interface.

2. Project Scope & Approach

2.1 Project Scope

The project implements the complete architecture proposed in the case study while reducing only the volume of processed data. Instead of attempting to collect information from hundreds of competitors and millions of documents, the platform focuses on a representative dataset that demonstrates every major software component and AI workflow.

The implementation includes real-time market data collection, Natural Language Processing, Knowledge Graph construction, semantic search using Retrieval-Augmented Generation (RAG), AI-powered reasoning, forecasting, caching, REST APIs, and an interactive Executive Dashboard.

The objective was to build a fully functional proof-of-concept that accurately represents how the platform would operate at enterprise scale.

2.2 Implemented Features

The following major components were successfully implemented during the project:

- Multi-source live data ingestion from public sources.
- PostgreSQL database for storing raw articles and customer reviews.
- Natural Language Processing pipeline using spaCy and TextBlob.

- Neo4j Knowledge Graph representing companies, products, events, articles, and reviews.
- Semantic Search using ChromaDB and Sentence Transformers.
- Time-series forecasting using Prophet.
- AI-powered SWOT Intelligence Generator.
- AI Strategy Advisor.
- Scenario Simulation Engine.
- AI Executive Copilot using LangGraph and Groq LLM.
- Explainable AI with supporting evidence and confidence scores.
- RESTful APIs developed using FastAPI.
- Redis caching for improved API performance.
- NetworkX-based competitive relationship analysis.
- Executive Dashboard for business visualization.
- Automated live-data update pipeline using Windows Task Scheduler.

2.3 Non-Implemented Features and Justification

Some data sources described in the original case study were not implemented because of technical, financial, or time-related limitations.

i. Company Websites and Product Catalogs

Each company maintains a different website structure without a common API. Developing reliable web scrapers for every company would require significant additional development effort, and the scrapers would require continuous maintenance whenever website structures changed. Therefore, these sources were excluded from the implementation.

ii. Social Media Integration

Public social media platforms impose restrictions on automated data collection. Twitter/X requires paid API access, while Reddit requires OAuth-based authentication and enforces strict rate limits. Due to these limitations and the available project timeline, social media data collection was not included.

These exclusions affect only the quantity of collected data and do not change the overall architecture or functionality of the platform.

2.4 Future Scalability

The platform has been designed using a modular architecture that supports future expansion without significant architectural changes.

Future improvements may include:

- Integration with additional commercial data providers.
- Company-specific website crawlers.
- Social media data collection through authenticated APIs.
- Expansion to hundreds of companies and product categories.
- Distributed data processing for large-scale deployments.
- Cloud-based deployment with containerization technologies.
- Real-time streaming using Apache Kafka or similar messaging platforms.
- Advanced predictive analytics using larger historical datasets.
- Multi-user authentication and role-based access control.

The current implementation provides a solid foundation for future enterprise-scale deployment while demonstrating the complete functionality of the proposed architecture.

3. System Architecture

3.1 Architectural Overview

The platform architecture is divided into the following five layers:

1. Raw Data Layer
2. Processing Layer
3. Knowledge Layer
4. AI Reasoning Layer
5. Presentation Layer

The interaction between these layers enables continuous data collection, intelligent processing, AI reasoning, and business visualization.

3.1.1 Layer 1: Raw Data Layer

The Raw Data Layer is responsible for collecting market intelligence from multiple publicly available information sources. The collected information includes business news, financial reports, patent information, customer reviews, and industry publications.

The platform currently retrieves data from the following sources:

- NewsAPI
- TechCrunch
- The Verge
- Ars Technica
- Engadget
- SEC EDGAR Financial Reports
- Google Patents

The collected information is initially stored in PostgreSQL before further processing.

3.1.2 Layer 2: Processing Layer

After collection, the data enters the processing layer where Natural Language Processing (NLP) techniques are applied.

This layer performs several preprocessing tasks including:

- Text cleaning
- Entity recognition
- Company identification
- Sentiment analysis
- Data normalization

The platform uses **spaCy** for Named Entity Recognition (NER) and company extraction, while **TextBlob** is used to calculate sentiment polarity for customer reviews.

The processed data is then forwarded to the Knowledge Layer.

3.1.3 Layer 3: Knowledge Layer

The Knowledge Layer organizes processed information into structured and searchable formats.

This layer consists of three major components:

PostgreSQL

Stores raw articles and customer reviews before processing.

Neo4j Knowledge Graph

Represents relationships between companies, products, market events, articles, and customer reviews using graph structures.

ChromaDB

Stores vector embeddings generated using Sentence Transformers, enabling semantic search through Retrieval-Augmented Generation (RAG).

Together, these databases provide structured storage, relationship analysis, and semantic information retrieval.

3.1.4 Layer 4: AI Reasoning Layer

The AI Reasoning Layer performs intelligent analysis using multiple specialized AI modules.

The implemented modules include:

- SWOT Intelligence Generator
- AI Strategy Advisor
- Scenario Simulation Engine
- Forecasting Engine
- AI Executive Copilot
- Explainable AI
- NetworkX Competitive Analysis

The platform uses **Groq-hosted Large Language Models (LLMs)** through LangGraph to generate strategic recommendations based on retrieved evidence from the Knowledge Graph and semantic search system.

Forecasting is performed using **Prophet**, while graph analytics are implemented using **NetworkX**.

Each AI module produces evidence-supported outputs together with confidence information wherever applicable.

3.1.5 Layer 5: Presentation Layer

The Presentation Layer provides users with an interactive interface for viewing market intelligence and AI-generated insights.

The platform includes an Executive Dashboard developed using HTML, CSS, and JavaScript. The dashboard communicates with the FastAPI backend through REST APIs.

The dashboard presents:

- Market Share
- Competitor Activity
- Product Sentiment
- Market Forecast
- Risk and Opportunity Indicators
- Innovation Index
- Strategic Recommendations

This layer enables executives and analysts to monitor market conditions and access AI-generated business intelligence through a simple web interface.

3.2 System Architecture Diagram

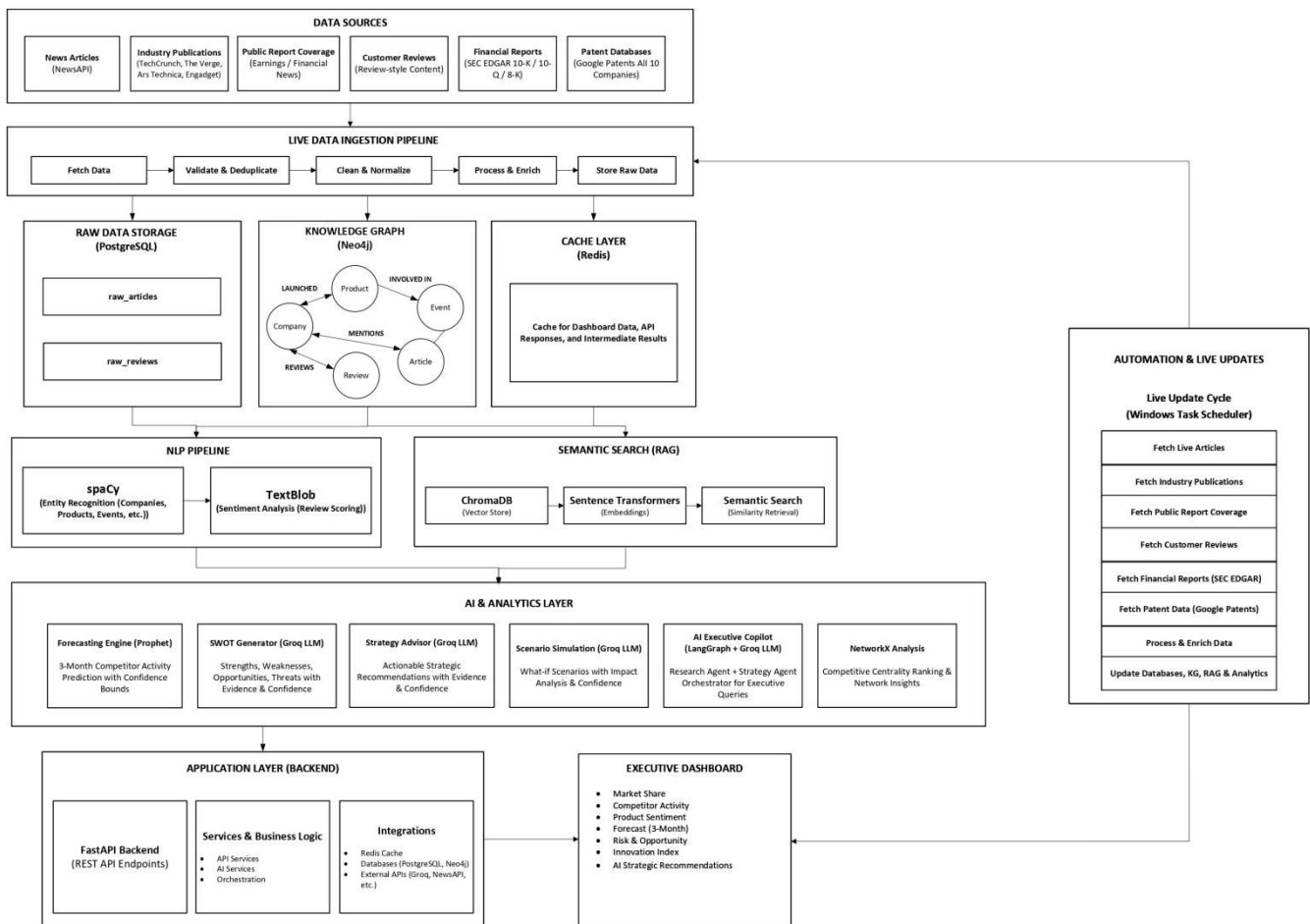


Figure 1: System Architecture Diagram

3.3 Data Flow

The platform processes information through a sequential pipeline consisting of the following stages:

1. Market information is collected from public data sources.
2. Raw data is stored in PostgreSQL.
3. The NLP pipeline extracts companies, products, events, and sentiment information.
4. Processed entities are inserted into the Neo4j Knowledge Graph.
5. Text embeddings are generated and stored in ChromaDB.
6. AI modules retrieve relevant information using semantic search and graph relationships.
7. Large Language Models generate strategic recommendations and business insights.
8. REST APIs deliver processed information to the Executive Dashboard.
9. The dashboard presents visual analytics and AI-generated recommendations to users.

This workflow enables continuous processing of market intelligence while maintaining a modular and scalable architecture.

4. Technology Stack

4.1 Backend Technologies

Table 1: Backend Technologies

Technology	Role in the Platform
Python	Primary implementation language across all services and scripts
FastAPI	REST API framework exposing every module as an HTTP endpoint
PostgreSQL 18	Durable storage for raw articles and reviews prior to processing
Neo4j	Knowledge graph database companies, products, events, regions, and their relationships
Redis (via Memurai on Windows)	Caching layer for expensive endpoints (dashboard, SWOT, Strategy Advisor, Forecast)

4.2 AI Technologies

Table 2: AI Technologies

Technology	Role in the Platform
LangGraph	LangGraph manages the workflow of multiple AI agents. It coordinates the interaction between the Research Agent and the Strategy Agent to generate comprehensive business recommendations.
LangChain	LangChain provides the framework for integrating Large Language Models into the platform. It simplifies prompt management and communication with external AI services.
ChromaDB	ChromaDB stores vector embeddings generated by Sentence Transformers. It serves as the vector database used by the Retrieval-Augmented Generation (RAG) system.
Sentence Transformers	Sentence Transformers convert textual information into numerical vector embeddings that preserve semantic meaning. These embeddings enable efficient similarity search.
Prophet	Prophet is used for time-series forecasting. It predicts future competitor activity based on historical market trends and provides confidence intervals for generated forecasts.

Groq	Groq-hosted Large Language Models perform advanced reasoning tasks such as SWOT analysis, strategy generation, scenario simulation, and executive question answering.
spaCy	spaCy is used for Natural Language Processing tasks such as Named Entity Recognition (NER). It extracts company names and other important entities from collected news articles.
TextBlob	TextBlob performs sentiment analysis on customer reviews. It calculates sentiment polarity, allowing the platform to identify whether customer feedback is positive, negative, or neutral.
NetworkX	NetworkX performs graph-based analysis on relationships stored within the Knowledge Graph. It calculates competitive centrality and helps identify influential companies within the market network.

4.3 Database Technologies

The platform uses multiple databases because different types of information require different storage models.

4.3.1 PostgreSQL

PostgreSQL stores raw market data before processing. It contains news articles, customer reviews, and related metadata collected during the live data ingestion process.

4.3.2 Neo4j

Neo4j stores the Knowledge Graph representing relationships between companies, products, market events, articles, and customer reviews. Graph databases allow efficient relationship analysis that would be difficult to perform using traditional relational databases.

4.3.3 Redis (Memurai)

Redis is used as an in-memory caching system to reduce API response time. Frequently requested dashboard information, SWOT results, strategy recommendations, and forecasting outputs are temporarily stored in Redis to improve system performance.

4.4 Development Tools

4.4.1 Visual Studio Code

Visual Studio Code served as the primary development environment for writing, debugging, and managing the project source code.

5. System Implementation

5.1 Multi-Source Data Intelligence

The platform collects market information from six live, publicly accessible sources: News Articles, Industry Publications, Public Report Coverage, Customer Reviews, Financial Reports, and Patent Databases. Each source is handled by an independent fetch script that writes into a shared PostgreSQL raw data store.

5.2 Live Data Ingestion Pipeline

Each of the six implemented sources has a dedicated script:

- **fetch_live_articles.py** general news articles via NewsAPI
- **fetch_industry_publications.py** coverage from TechCrunch, The Verge, Ars Technica, and Engadget, using NewsAPI's domain filter
- **fetch_public_reports.py** earnings and annual-report coverage
- **fetch_live_reviews.py** review-style content
- **fetch_financial_reports.py** official SEC EDGAR filings (10-K, 10-Q, 8-K) for Apple and Google, the only SEC-registered companies among the ten tracked
- **fetch_patent_data.py** patent search results from Google Patents' public search endpoint, covering all ten companies

Every fetch script performs URL-based deduplication before inserting a record, so repeated runs only add genuinely new items. A combined script, **live_update_cycle.py**, runs all six fetch scripts followed by the NLP processing scripts in sequence, and is registered as a Windows Task Scheduler job that executes automatically every six hours.

C:\Windows\system32\cmd.exe

```
(venv) C:\Users\Hassan\Downloads\market-intel-platform>python scripts\live_update_cycle.py  
=== Live update cycle started: 2026-08-15T09:48:28.189042 ===
```

[2026-08-15T09:48:28.189042] Running fetch_live_articles.py...

Samsung: fetched 5, inserted 0 new so far
Apple: fetched 5, inserted 0 new so far
Xiaomi: fetched 5, inserted 0 new so far
OnePlus: fetched 5, inserted 0 new so far
Google: fetched 5, inserted 0 new so far
Vivo: fetched 5, inserted 0 new so far
Oppo: fetched 5, inserted 0 new so far
Huawei: fetched 5, inserted 0 new so far
Sony: fetched 5, inserted 0 new so far
Motorola: fetched 5, inserted 0 new so far

Done. 0 new live articles inserted into raw_articles.

[2026-08-15T09:48:38.390769] Running fetch_industry_publications.py...

Samsung: fetched 3, inserted 0 new
Apple: fetched 3, inserted 0 new
Xiaomi: fetched 1, inserted 0 new
OnePlus: fetched 1, inserted 0 new
Google: fetched 3, inserted 0 new
Vivo: fetched 3, inserted 0 new
Oppo: fetched 1, inserted 0 new
Huawei: fetched 3, inserted 0 new
Sony: fetched 1, inserted 0 new
Motorola: fetched 2, inserted 0 new

Done. 0 new industry-publication articles inserted.

[2026-08-15T09:48:48.056945] Running fetch_public_reports.py...

Samsung: fetched 3, inserted 0 new
Apple: fetched 3, inserted 0 new
Xiaomi: fetched 3, inserted 0 new
OnePlus: fetched 3, inserted 0 new
Google: fetched 3, inserted 0 new
Vivo: fetched 3, inserted 0 new
Oppo: fetched 3, inserted 0 new
Huawei: fetched 3, inserted 0 new
Sony: fetched 3, inserted 0 new
Motorola: fetched 3, inserted 0 new

Done. 0 new public-report-coverage articles inserted.

[2026-08-15T09:48:57.909861] Running fetch_financial_reports.py...

Apple: found 5 recent filings, inserted 0 new
Google: found 5 recent filings, inserted 0 new

Skipped (not SEC-registered / not covered by this endpoint): Samsung, Xiaomi, OnePlus, Vivo, Oppo, Huawei, Sony, Motorola
Done. 0 new SEC filing records inserted into raw_articles.

[2026-08-15T09:49:05.128100] Running fetch_patent_data.py...

Samsung: fetch failed (503 Server Error: Service Unavailable for url: <https://patents.google.com/xhr/query?url=q%3DSamsung%2Bsmartphone&exp=&content=1>)
Apple: fetch failed (503 Server Error: Service Unavailable for url: <https://patents.google.com/xhr/query?url=q%3DApple%2Bsmartphone&exp=&content=1>)
Xiaomi: fetch failed (503 Server Error: Service Unavailable for url: <https://patents.google.com/xhr/query?url=q%3DXiaomi%2Bsmartphone&exp=&content=1>)
OnePlus: fetch failed (503 Server Error: Service Unavailable for url: <https://patents.google.com/xhr/query?url=q%3DOnePlus%2Bsmartphone&exp=&content=1>)
Google: fetch failed (503 Server Error: Service Unavailable for url: <https://patents.google.com/xhr/query?url=q%3DGoogle%2Bsmartphone&exp=&content=1>)
Vivo: fetch failed (503 Server Error: Service Unavailable for url: <https://patents.google.com/xhr/query?url=q%3DVivo%2Bsmartphone&exp=&content=1>)
Oppo: fetch failed (503 Server Error: Service Unavailable for url: <https://patents.google.com/xhr/query?url=q%3DOppo%2Bsmartphone&exp=&content=1>)
Huawei: fetch failed (503 Server Error: Service Unavailable for url: <https://patents.google.com/xhr/query?url=q%3DHuawei%2Bsmartphone&exp=&content=1>)
Sony: fetch failed (503 Server Error: Service Unavailable for url: <https://patents.google.com/xhr/query?url=q%3DSony%2Bsmartphone&exp=&content=1>)
Motorola: fetch failed (503 Server Error: Service Unavailable for url: <https://patents.google.com/xhr/query?url=q%3DMotorola%2Bsmartphone&exp=&content=1>)

Done. 0 new patent records inserted into raw_articles.

[2026-08-15T09:49:19.814990] Running process_articles.py...

Article 419: found companies {'Samsung', 'Google'} -> marked processed.
Article 420: found companies none -> marked processed.
Article 421: found companies {'Samsung'} -> marked processed.
Article 422: found companies {'Samsung', 'Google'} -> marked processed.
Article 423: found companies {'Apple'} -> marked processed.
Article 424: found companies none -> marked processed.
Article 425: found companies none -> marked processed.
Article 426: found companies none -> marked processed.
Article 427: found companies {'Xiaomi'} -> marked processed.
Article 428: found companies none -> marked processed.
Article 429: found companies none -> marked processed.
Article 430: found companies {'Google'} -> marked processed.
Article 431: found companies {'Google'} -> marked processed.

Article 432: found companies {'Google'} -> marked processed.
Article 433: found companies {'Google'} -> marked processed.
Article 434: found companies {'Apple', 'Google'} -> marked processed.
Article 435: found companies {'Google'} -> marked processed.
Article 436: found companies none -> marked processed.
Article 437: found companies {'Google'} -> marked processed.
Article 438: found companies {'Google'} -> marked processed.
Article 439: found companies none -> marked processed.
Article 440: found companies none -> marked processed.
Article 441: found companies none -> marked processed.
Article 442: found companies none -> marked processed.
Article 443: found companies none -> marked processed.
Article 444: found companies {'Apple'} -> marked processed.
Article 445: found companies none -> marked processed.
Article 446: found companies none -> marked processed.
Article 447: found companies none -> marked processed.
Article 448: found companies none -> marked processed.
Article 449: found companies {'Samsung'} -> marked processed.
Article 450: found companies none -> marked processed.
Article 451: found companies {'Apple'} -> marked processed.
Article 452: found companies none -> marked processed.
Article 453: found companies {'Samsung'} -> marked processed.
Article 454: found companies none -> marked processed.
Article 455: found companies none -> marked processed.
Article 456: found companies {'Apple'} -> marked processed.
Article 457: found companies {'Google'} -> marked processed.
Article 458: found companies {'Apple'} -> marked processed.
Article 459: found companies none -> marked processed.
Article 460: found companies none -> marked processed.
Article 461: found companies none -> marked processed.
Article 462: found companies none -> marked processed.
Article 463: found companies none -> marked processed.
Article 464: found companies none -> marked processed.
Article 465: found companies none -> marked processed.
Article 466: found companies none -> marked processed.
Article 467: found companies {'Apple'} -> marked processed.
Article 468: found companies none -> marked processed.
Article 469: found companies none -> marked processed.
Article 470: found companies none -> marked processed.
Article 471: found companies none -> marked processed.
Article 472: found companies none -> marked processed.
Article 473: found companies none -> marked processed.
Article 474: found companies none -> marked processed.

```

Article 475: found companies none -> marked processed.

57 articles processed.
Article processing complete.

[2026-08-15T09:50:06.435925] Running fetch_live_reviews.py...
Galaxy S25: fetched 3, inserted 0 new
Galaxy A55: fetched 0, inserted 0 new
iPhone 16: fetched 3, inserted 0 new
iPhone 16e: fetched 0, inserted 0 new
Xiaomi 15: fetched 0, inserted 0 new
Redmi Note 14: fetched 3, inserted 0 new
OnePlus 13: fetched 3, inserted 0 new
Pixel 9: fetched 3, inserted 0 new
Vivo X200: fetched 1, inserted 0 new
Vivo Y29: fetched 0, inserted 0 new
Oppo Find X8: fetched 0, inserted 0 new
Oppo A3: fetched 0, inserted 0 new
Huawei Pura 70: fetched 0, inserted 0 new
Sony Xperia 1 VI: fetched 0, inserted 0 new
Motorola Edge 50: fetched 1, inserted 0 new

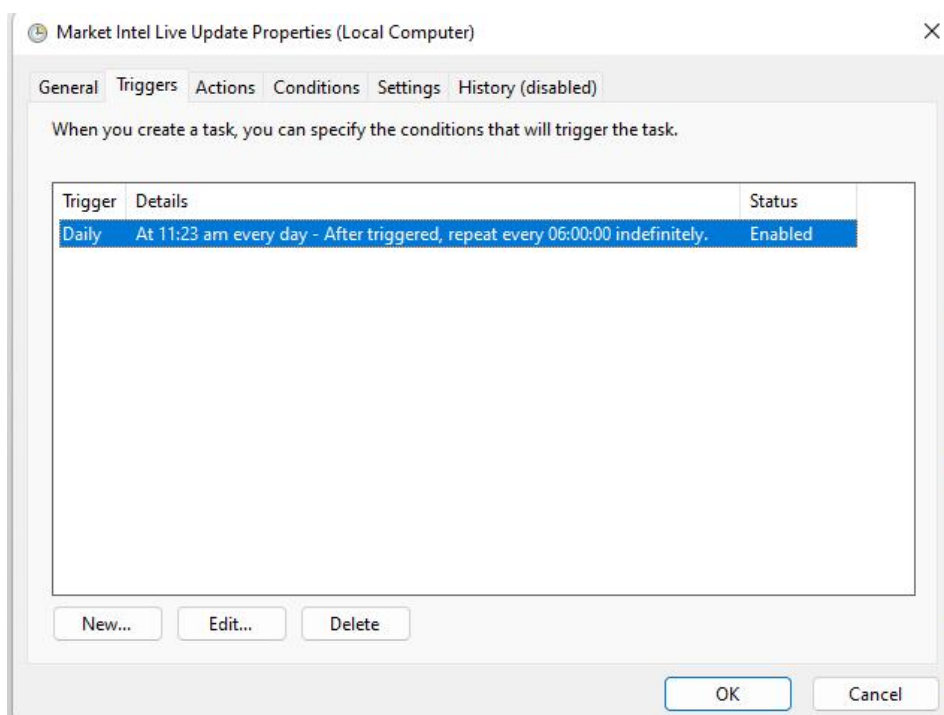
Done. 0 new live review-style items inserted into raw_reviews.

[2026-08-15T09:50:20.407637] Running process_reviews.py...
Review 41: product=Galaxy S25, sentiment=0.45 -> marked processed.
Review 42: product=Galaxy S25, sentiment=0.333 -> marked processed.
Review 43: product=Galaxy S25, sentiment=1.0 -> marked processed.
Review 44: product=iPhone 16, sentiment=0.042 -> marked processed.
Review 45: product=iPhone 16, sentiment=-0.2 -> marked processed.
Review 46: product=Pixel 9, sentiment=0.357 -> marked processed.
Review 47: product=Pixel 9, sentiment=0.0 -> marked processed.

7 reviews processed.
Review processing complete.

=== Cycle complete ===
fetch_live_articles.py: OK
fetch_industry_publications.py: OK
fetch_public_reports.py: OK
fetch_financial_reports.py: OK
fetch_patent_data.py: OK
process_articles.py: OK
fetch_live_reviews.py: OK
process_reviews.py: OK

```

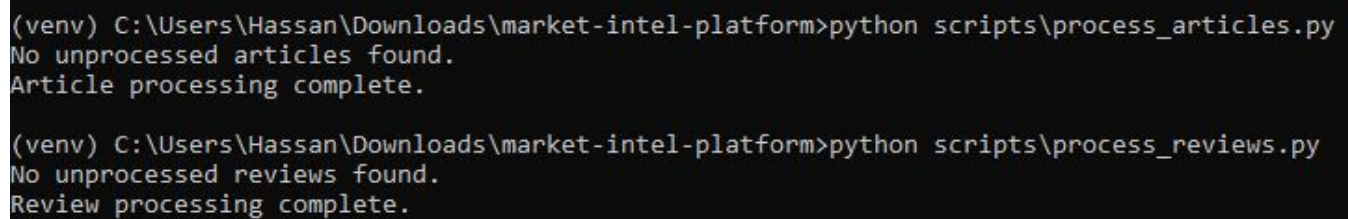


5.3 PostgreSQL Raw Data Storage

Raw articles and reviews are stored in two PostgreSQL tables, **raw_articles** and **raw_reviews**, prior to any analysis. Each row includes a processed flag, set to false on insertion and updated to true once the NLP pipeline has extracted information from it. This separation keeps raw source material intact and allows the processing pipeline to run repeatedly without reprocessing the same data.

5.4 NLP Pipeline

Two natural language processing steps convert raw text into structured facts. spaCy's named-entity recognition identifies company mentions in article text, matched against the list of tracked company names. TextBlob computes a sentiment polarity score, from -1.0 to +1.0, for each review. Both steps write their results into the knowledge graph and mark the corresponding PostgreSQL row as processed.

A terminal window with a black background and white text. It shows two commands being executed. The first command is 'python scripts\process_articles.py' and the output is 'No unprocessed articles found. Article processing complete.' The second command is 'python scripts\process_reviews.py' and the output is 'No unprocessed reviews found. Review processing complete.'

```
(venv) C:\Users\Hassan\Downloads\market-intel-platform>python scripts\process_articles.py
No unprocessed articles found.
Article processing complete.

(venv) C:\Users\Hassan\Downloads\market-intel-platform>python scripts\process_reviews.py
No unprocessed reviews found.
Review processing complete.
```

Figure 2: NLP Pipeline

5.5 Knowledge Graph (Neo4j)

The knowledge graph stores six node types: Company, Product, Event, Region, Article, and Review, connected by four relationship types: **LAUNCHED** (Company to Product), **INVOLVED_IN** (Company to Event), **MENTIONS** (Article to Company), and **REVIEWS** (Review to Product). The graph currently holds ten companies, fifteen products, and market events covering price changes, geographic expansion, partnerships, and technology adoption.

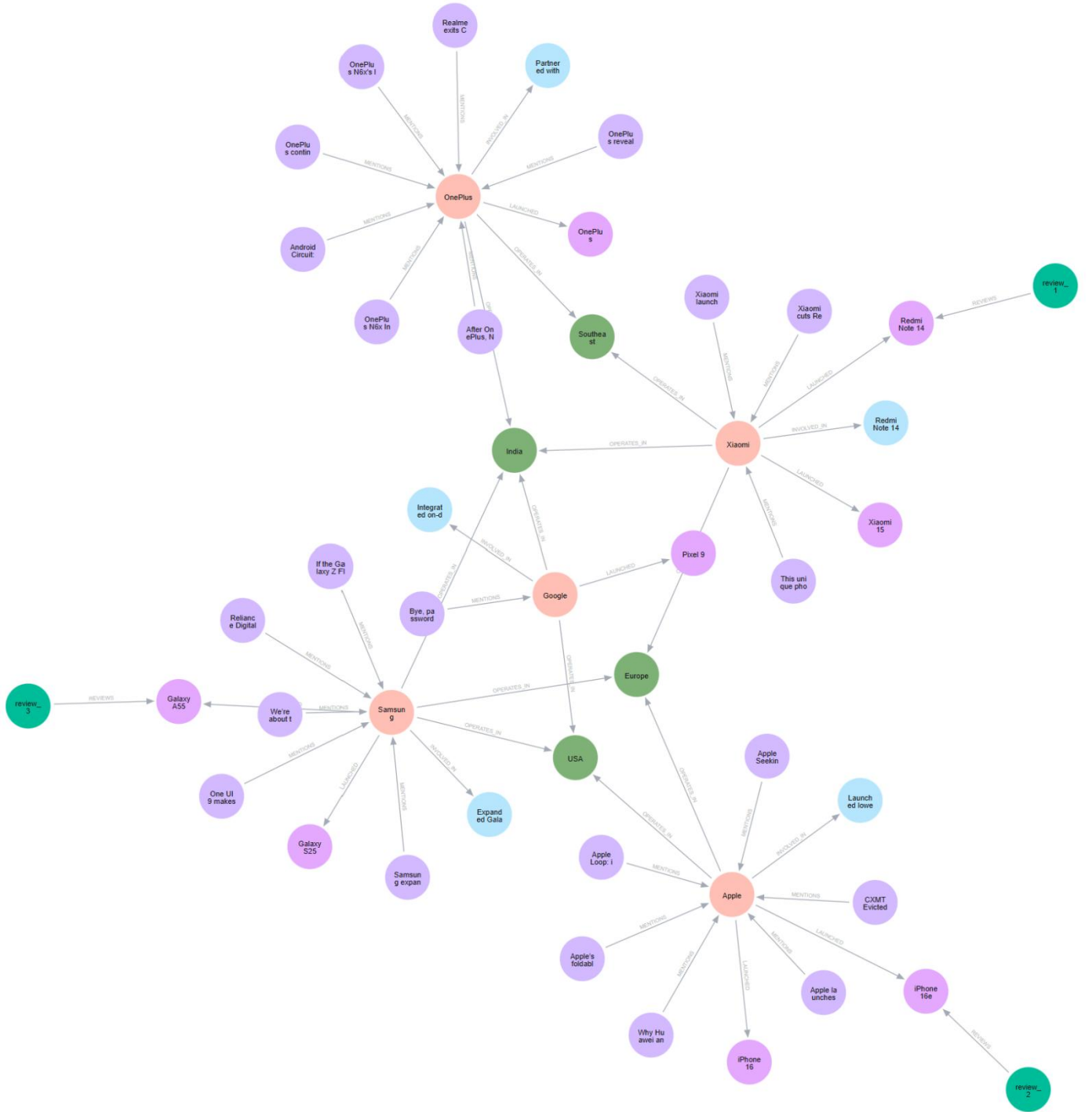


Figure 3: Knowledge Graph (Neo4j)

5.6 Semantic Search (RAG)

Processed articles and reviews are embedded using the Sentence Transformers model **all-MiniLM-L6-v2** and stored in a persistent ChromaDB collection. This enables retrieval by meaning rather than exact keyword match, verified during testing by a query that returned relevant results despite sharing no exact words with the underlying source text.


```
(venv) C:\Users\Hassan\Downloads\market-intel-platform>python scripts\test_rag_search.py
Failed to send telemetry event ClientStartEvent: capture() takes 1 positional argument but 3 were given
'(ReadTimeoutError("HTTPSConnectionPool(host='huggingface.co', port=443): Read timed out. (read timeout=10)"), '(Request ID: 5fb4d10b-f40a-4e57-a061-7968ce316293)')' thrown while requesting HEAD https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/resolve/main/modules.json
Retrying in 1s [Retry 1/5].
Failed to send telemetry event ClientCreateCollectionEvent: capture() takes 1 positional argument but 3 were given
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument but 3 were given

--- Result 1 ---
Type: article
Metadata: {'company': 'Xiaomi', 'date': '2025-04-01', 'title': 'Xiaomi cuts Redmi Note 14 price in India', 'type': 'article'}
Text: Xiaomi cuts Redmi Note 14 price in India. Xiaomi has reduced the price of its Redmi Note 14 by 10% in the Indian market amid heightened competition fr...

--- Result 2 ---
Type: article
Metadata: {'company': 'Apple', 'date': '2025-02-19', 'title': 'Apple launches iPhone 16e as budget alternative', 'type': 'article'}
Text: Apple launches iPhone 16e as budget alternative. Apple unveiled the iPhone 16e, a lower-priced model aimed at competing with budget Android manufactur...

--- Result 3 ---
Type: review
Metadata: {'date': '2025-03-01', 'product': 'iPhone 16e', 'rating': 4.0, 'type': 'review'}
Text: Good performance for the price, but I wish it had a better display refresh rate....
```

Figure 4: Semantic Search (RAG)

5.7 Forecasting Engine

The forecasting module uses Prophet to build a monthly time series of event counts per company, or across all companies, and project three months forward with upper and lower confidence bounds. When fewer than two months of history exist for a given company, the module returns a status of **insufficient_data** rather than producing an unreliable projection.

Response body

```
{
  "status": "ok",
  "company": "all_competitors",
  "history_months": 4,
  "forecast": [
    {
      "month": "2025-05",
      "predicted_activity": 2.5,
      "lower_bound": 1.87,
      "upper_bound": 3.17
    },
    {
      "month": "2025-06",
      "predicted_activity": 2.5,
      "lower_bound": 1.84,
      "upper_bound": 3.16
    },
    {
      "month": "2025-07",
      "predicted_activity": 2.5,
      "lower_bound": 1.88,
      "upper_bound": 3.11
    }
  ],
  "_cache": "hit"
}
```

Figure 5: Forecasting Engine

5.8 SWOT Intelligence Generator

For a given company, this module gathers its products, related market events, and review sentiment from the knowledge graph, then prompts a large language model (Groq (OpenAI GPT-OSS 120B)) to produce a structured Strengths, Weaknesses, Opportunities, and Threats analysis. Each point is required to cite specific evidence, and the module returns an overall confidence level.

```
{
```

```

"status": "ok",
"company": "Apple",
"strengths": [
  {
    "point": "Strong ecosystem integration and innovative charging solutions",
    "evidence": "Review mentions \"iPhone/AirPods/Watch charge simultaneously\", \"25W iPhone charging\", \"StealthCharge wireless charging tech\", and \"Smart travel case\" for iPhone 16."
  },
  {
    "point": "Positive reception of AI enhancements in Siri",
    "evidence": "Multiple review signals for iPhone 16 show positive sentiment about Siri AI (sentiments 0.127, 0.05, 0.133, 0.084)."
  },
  {
    "point": "Effective financing partnership expanding affordability",
    "evidence": "Customer review signal notes Apple partnering with Klarna to offer device leases with low monthly payments."
  },
  {
    "point": "Budget flagship offering expands market reach",
    "evidence": "Product line includes iPhone 16e priced at $599 and market event \"Launched lower-priced iPhone 16e to counter budget competitors\"."
  },
  {
    "point": "Good value perception for budget model",
    "evidence": "iPhone 16e received a 4.0 rating and sentiment 0.6, praising performance for price."
  }
],
"weaknesses": [
  {
    "point": "Mixed/negative consumer sentiment on flagship experience",
    "evidence": "Several iPhone 16 review signals have neutral to negative sentiment (e.g., -0.2 about cheap PCs, -0.2 about security feature misrepresentation, and complaints about Siri faults)."
  },
  {
    "point": "High flagship price may limit competitiveness",
    "evidence": "iPhone 16 listed at $999, while a budget competitor (Samsung Galaxy Tab S10 FE) is highlighted for better value."
  },
  {
    "point": "Camera performance lagging behind newer models",
    "evidence": "Review notes iPhone 16 ranks 20 places behind iPhone 17 in DxOMark camera analysis."
  },
  {
    "point": "Display refresh rate concerns on budget model",
    "evidence": "iPhone 16e review: \"Good performance for the price, but I wish it had a better display refresh rate.\""
  }
]

```

```

    "point": "Legal and security reputational risk",
    "evidence": "Review mentions a $250 million settlement lawsuit alleging
misrepresentation of a key security feature."
  },
],
"opportunities": [
  {
    "point": "Capture price-sensitive segment with iPhone 16e",
    "evidence": "Market event: launch of lower-priced iPhone 16e to counter budget
competitors."
  },
  {
    "point": "Leverage device-leasing to increase adoption",
    "evidence": "Partnership with Klarna for low-monthly-payment leases mentioned in
review signals."
  },
  {
    "point": "Differentiate via advanced Siri AI capabilities",
    "evidence": "Positive sentiment around Siri AI upgrades (sentiments up to 0.133)
suggests a competitive edge."
  },
  {
    "point": "Expand ecosystem with upcoming Apple Watch refresh and Mac overhaul",
    "evidence": "Review signal references \"refreshed Apple Watch models\" and \"full
Mac overhaul\" as upcoming initiatives."
  },
  {
    "point": "Improve camera and display specs to close gaps with competitors",
    "evidence": "Negative sentiment about camera lag and display refresh rate highlights
clear product improvement areas."
  }
],
"threats": [
  {
    "point": "Intense competition from Android manufacturers offering better battery,
display, and price",
    "evidence": "Review compares Samsung Galaxy Tab S10 FE favorably on battery life and
display; mention of Samsung Galaxy S26 Ultra in camera comparison."
  },
  {
    "point": "Legal challenges and potential settlements",
    "evidence": "Reference to a $250 million settlement lawsuit alleging
misrepresentation of a security feature."
  },
  {
    "point": "Consumer perception of high price may drive churn to cheaper alternatives",
    "evidence": "Negative sentiment about \"cheap PCs\" and the need for budget
options."
  },
  {
    "point": "Rapid innovation cycles from competitors could erode Apple's premium
advantage",

```

```

    "evidence": "Mentions of newer models (iPhone 17, Samsung S26 Ultra) outperforming
iPhone 16."
  },
],
"confidence": "medium",
"_cache": "hit"
}

```

5.9 AI Strategy Advisor

This module recommends actions across up to six categories: New Market Entry, Pricing Strategy, Product Roadmap, Partnership Opportunities, Investment Priorities, and Competitive Response. It follows the same evidence-gathering and prompting pattern as the SWOT generator, and omits any category the available data does not reasonably support.

```

{
  "status": "ok",
  "company": "Apple",
  "recommendations": [
    {
      "category": "Pricing Strategy",
      "recommendation": "Introduce a modest price reduction for the iPhone 16 flagship
(e.g., $950) or add a mid-tier model priced around $799 to better compete with Google
Pixel 9 and OnePlus 13.",
      "rationale": "The iPhone 16 flagship sentiment is low (0.05) while competitors offer
comparable or superior specs at lower prices (Pixel 9 $799, OnePlus 13 $899). A price
adjustment can improve perception and capture price-sensitive premium buyers.",
      "evidence": "iPhone 16 sentiment 0.049 vs. flagship competitors priced $799-$949;
recent competitor price cuts in budget segment indicate market sensitivity to price.",
      "priority": "high"
    },
    {
      "category": "New Market Entry",
      "recommendation": "Accelerate retail and carrier partnerships for the iPhone 16e in
Southeast Asia and South Asia, focusing on high-traffic malls and online channels.",
      "rationale": "Samsung and Vivo have recently expanded retail presence in Southeast
and South Asia, respectively, increasing competition in those growth markets. The iPhone
16e shows strong sentiment (0.30) and a solid rating (4.0), indicating demand for a budget
Apple offering.",
      "evidence": "Samsung geographic expansion (2025-03-20) and Vivo expansion (2025-03-
15); iPhone 16e sentiment 0.301 and rating 4.0; competitor budget price cuts in the region
(Oppo A3, Xiaomi Redmi Note 14).",
      "priority": "high"
    },
    {
      "category": "Competitive Response",
      "recommendation": "Launch limited-time bundled promotions (e.g., free Apple TV+ trial
+ carrier data plan) for both iPhone 16 and 16e to counter recent budget price cuts by
Motorola, Oppo, and Xiaomi.",

```



```

    "rationale": "Competitors are aggressively cutting prices in the budget segment (Motorola Edge 50 -12%, Oppo A3 -8%, Redmi Note 14 -10%). Bundling services adds value without eroding margin and can shift consumer preference toward Apple.",
    "evidence": "Recent price_change events for Motorola, Oppo, Xiaomi; OnePlus partnership with telecom for bundled data plans (2025-02-10) shows effectiveness of this tactic.",
    "priority": "medium"
  },
  {
    "category": "Product Roadmap",
    "recommendation": "Integrate on-device AI accelerators and advanced camera sensor technology into the next iPhone generation, leveraging lessons from Google's AI rollout and Sony's sensor partnership.",
    "rationale": "Google's on-device AI (2025-01-20) and Sony's exclusive imaging tech partnership (2025-02-15) are differentiators that can justify premium pricing and improve user sentiment, which is currently low for the flagship.",
    "evidence": "Google technology adoption event; Sony partnership event; low iPhone 16 sentiment suggests need for feature differentiation.",
    "priority": "medium"
  },
  {
    "category": "Partnership Opportunities",
    "recommendation": "Form strategic alliances with regional telecom operators in emerging markets to offer bundled data plans and device financing for the iPhone 16e.",
    "rationale": "OnePlus's recent telecom partnership (2025-02-10) demonstrates the market impact of such bundles, especially in price-sensitive regions where budget competitors are gaining share.",
    "evidence": "OnePlus partnership event; competitor geographic expansions; strong sentiment for iPhone 16e indicating readiness for market penetration.",
    "priority": "low"
  }
],
"confidence": "high",
"confidence_reason": "Recommendations are directly derived from explicit market data points, price comparisons, sentiment scores, and documented competitor actions.",
"_cache": "hit"
}

```

5.10 Scenario Simulation Engine

Given a free-text hypothetical scenario, such as a competitor price reduction, and an optional focal company, this module gathers the current competitive landscape and prompts the language model to predict likely business outcomes across several impact areas, each with a severity rating, alongside an overall confidence level and a recommended response.

```

{
  "status": "ok",
  "scenario": "string",
  "predicted_impacts": [
    {
      "area": "Pricing Pressure",

```

```

    "prediction": "Budget-segment price cuts by Xiaomi, Oppo, and Motorola will compress average selling prices and force other low-mid players to consider discounts or value-added bundles.",
    "reasoning": "Recent 8-10% cuts in India and Southeast Asia for Redmi Note 14 and Oppo A3, plus a 12% cut for Motorola Edge 50, directly lower the price floor in the $150-$500 range, eroding margin for comparable Samsung A55, Vivo Y29 and other mid-range devices.",
    "severity": "high"
  },
  {
    "area": "Market Share Shift",
    "prediction": "Xiaomi and Oppo are likely to gain modest share in emerging markets, while Samsung's Southeast Asian retail expansion may offset some loss in the A-series segment.",
    "reasoning": "Geographic expansion of Samsung A-series and Vivo X-series increases distribution, but price-driven promotions from Xiaomi and Oppo target the same consumer base, creating a competitive tug-of-war for share.",
    "severity": "medium"
  },
  {
    "area": "Premium Segment Dynamics",
    "prediction": "Apple's introduction of the lower-priced iPhone 16e will protect its premium brand perception while attracting price-sensitive buyers, limiting upside for Sony and Huawei in the $900-$1100 tier.",
    "reasoning": "Apple maintains a $999 flagship price but adds a $599 entry-point, which may siphon customers who would otherwise consider Sony Xperia 1 VI ($1099) or Huawei Pura 70 ($899) for premium features.",
    "severity": "medium"
  },
  {
    "area": "Differentiation via Technology",
    "prediction": "Huawei's AI chipset integration and Google's on-device AI features will provide modest differentiation but are unlikely to translate into immediate share gains without complementary pricing or carrier deals.",
    "reasoning": "Both tech adoptions are recent (Jan 2025) and target premium perception; however, price sensitivity in key growth markets remains dominant, limiting short-term impact.",
    "severity": "low"
  },
  {
    "area": "Competitor Response",
    "prediction": "OnePlus and Vivo are expected to accelerate bundled data or service offers to counter price cuts, potentially leading to short-term sales spikes in their respective regions.",
    "reasoning": "OnePlus already partnered with a regional telecom for bundled plans; Vivo's South Asia expansion suggests a similar strategy to boost volume without direct price reductions.",
    "severity": "medium"
  }
],
"recommended_response": [
  "Review mid-range pricing strategy; consider limited-time discounts or value bundles for Samsung Galaxy A55 and Vivo Y29 to stay competitive.",

```

```

    "Accelerate carrier and service bundling (e.g., data, warranty) for flagship and
    mid-range models to offset pure price competition.",
    "Invest in differentiated AI and imaging features across the portfolio and highlight
    them in marketing to counter Huawei and Sony tech moves.",
    "Monitor sales data in Southeast Asia and India closely; be prepared to adjust retail
    footprint or promotional spend in response to competitor geographic expansions."
  ],
  "confidence": "medium",
  "confidence_reason": "Predictions are based on recent price changes, geographic
  expansions, and partnership announcements, but lack detailed sales volume or consumer
  sentiment data, limiting precision."
}

```

5.11 AI Executive Copilot

The Executive Copilot answers free-form questions about the market. It is implemented as a two-node LangGraph state machine: a Research node that retrieves relevant knowledge graph evidence, and a Strategy node that reasons over that evidence to produce an answer.

```

{
  "question": "Which competitor is growing the fastest?",
  "answer": "The data provided does not contain direct information on the growth rates of
  the competitors. However, we can infer potential growth indicators from market events such
  as geographic expansions, which may indicate increasing market presence. Based on this,
  companies like Samsung and Vivo have recently expanded their retail presence, which could
  be a sign of growth. Confidence score: 20. \n\nThe data is insufficient to accurately
  determine which competitor is growing the fastest, as it lacks specific growth metrics
  such as revenue or sales data.",
  "evidence_used": {
    "companies": [
      {
        "name": "Samsung",
        "type": "competitor",
        "headquarters": "South Korea"
      },
      {
        "name": "Apple",
        "type": "competitor",
        "headquarters": "USA"
      },
      {
        "name": "Xiaomi",
        "type": "competitor",
        "headquarters": "China"
      },
      {
        "name": "OnePlus",
        "type": "competitor",
        "headquarters": "China"
      },
      {
        "name": "Google",

```

```

    "type": "competitor",
    "headquarters": "USA"
  },
  {
    "name": "Vivo",
    "type": "competitor",
    "headquarters": "China"
  },
  {
    "name": "Oppo",
    "type": "competitor",
    "headquarters": "China"
  },
  {
    "name": "Huawei",
    "type": "competitor",
    "headquarters": "China"
  },
  {
    "name": "Sony",
    "type": "competitor",
    "headquarters": "Japan"
  },
  {
    "name": "Motorola",
    "type": "competitor",
    "headquarters": "USA"
  }
],
"market_events": [
  {
    "company": "Oppo",
    "type": "price_change",
    "description": "Oppo A3 price reduced by 8% in Southeast Asia",
    "date": {
      "_Date__ordinal": 739346,
      "_Date__year": 2025,
      "_Date__month": 4,
      "_Date__day": 5
    }
  },
  {
    "company": "Xiaomi",
    "type": "price_change",
    "description": "Redmi Note 14 price cut by 10% in India",
    "date": {
      "_Date__ordinal": 739342,
      "_Date__year": 2025,
      "_Date__month": 4,
      "_Date__day": 1
    }
  }
],
{

```

```

"company": "Motorola",
"type": "price_change",
"description": "Edge 50 price cut by 12% ahead of new model launch",
"date": {
  "_Date__ordinal": 739338,
  "_Date__year": 2025,
  "_Date__month": 3,
  "_Date__day": 28
}
},
{
  "company": "Samsung",
  "type": "geographic_expansion",
  "description": "Expanded Galaxy A-series retail presence in Southeast Asia",
  "date": {
    "_Date__ordinal": 739330,
    "_Date__year": 2025,
    "_Date__month": 3,
    "_Date__day": 20
  }
},
{
  "company": "Vivo",
  "type": "geographic_expansion",
  "description": "Expanded Vivo X-series retail presence across South Asia",
  "date": {
    "_Date__ordinal": 739325,
    "_Date__year": 2025,
    "_Date__month": 3,
    "_Date__day": 15
  }
},
{
  "company": "Apple",
  "type": "price_change",
  "description": "Launched lower-priced iPhone 16e to counter budget competitors",
  "date": {
    "_Date__ordinal": 739301,
    "_Date__year": 2025,
    "_Date__month": 2,
    "_Date__day": 19
  }
},
{
  "company": "Sony",
  "type": "partnership",
  "description": "Partnered with a camera sensor supplier for exclusive imaging tech",
  "date": {
    "_Date__ordinal": 739297,
    "_Date__year": 2025,
    "_Date__month": 2,
    "_Date__day": 15
  }
}

```

```

    },
    {
      "company": "OnePlus",
      "type": "partnership",
      "description": "Partnered with a regional telecom for bundled data plans",
      "date": {
        "_Date__ordinal": 739292,
        "_Date__year": 2025,
        "_Date__month": 2,
        "_Date__day": 10
      }
    },
    {
      "company": "Huawei",
      "type": "technology_adoption",
      "description": "Integrated proprietary AI chipset across Pura series",
      "date": {
        "_Date__ordinal": 739276,
        "_Date__year": 2025,
        "_Date__month": 1,
        "_Date__day": 25
      }
    },
    {
      "company": "Google",
      "type": "technology_adoption",
      "description": "Integrated on-device AI features across Pixel lineup",
      "date": {
        "_Date__ordinal": 739271,
        "_Date__year": 2025,
        "_Date__month": 1,
        "_Date__day": 20
      }
    }
  ]
}

```

5.12 Explainable AI

Every module that generates an AI-based conclusion, the SWOT generator, Strategy Advisor, Scenario Simulation engine, and Executive Copilot returns three components without exception: the conclusion itself, the specific evidence used, and a confidence level. Confidence is permitted to be low, and the system explicitly states when available data does not support a confident answer, rather than presenting an estimate as certain.

5.13 Executive Dashboard

The Executive Dashboard aggregates data from every other module into a single view, implemented as a FastAPI endpoint and a static HTML and JavaScript frontend using Chart.js. It displays Market Share, Competitor Activity, Product Sentiment, a three-month Activity Forecast, Risk and Opportunity scores, an Innovation Index, and AI Strategic Recommendations.

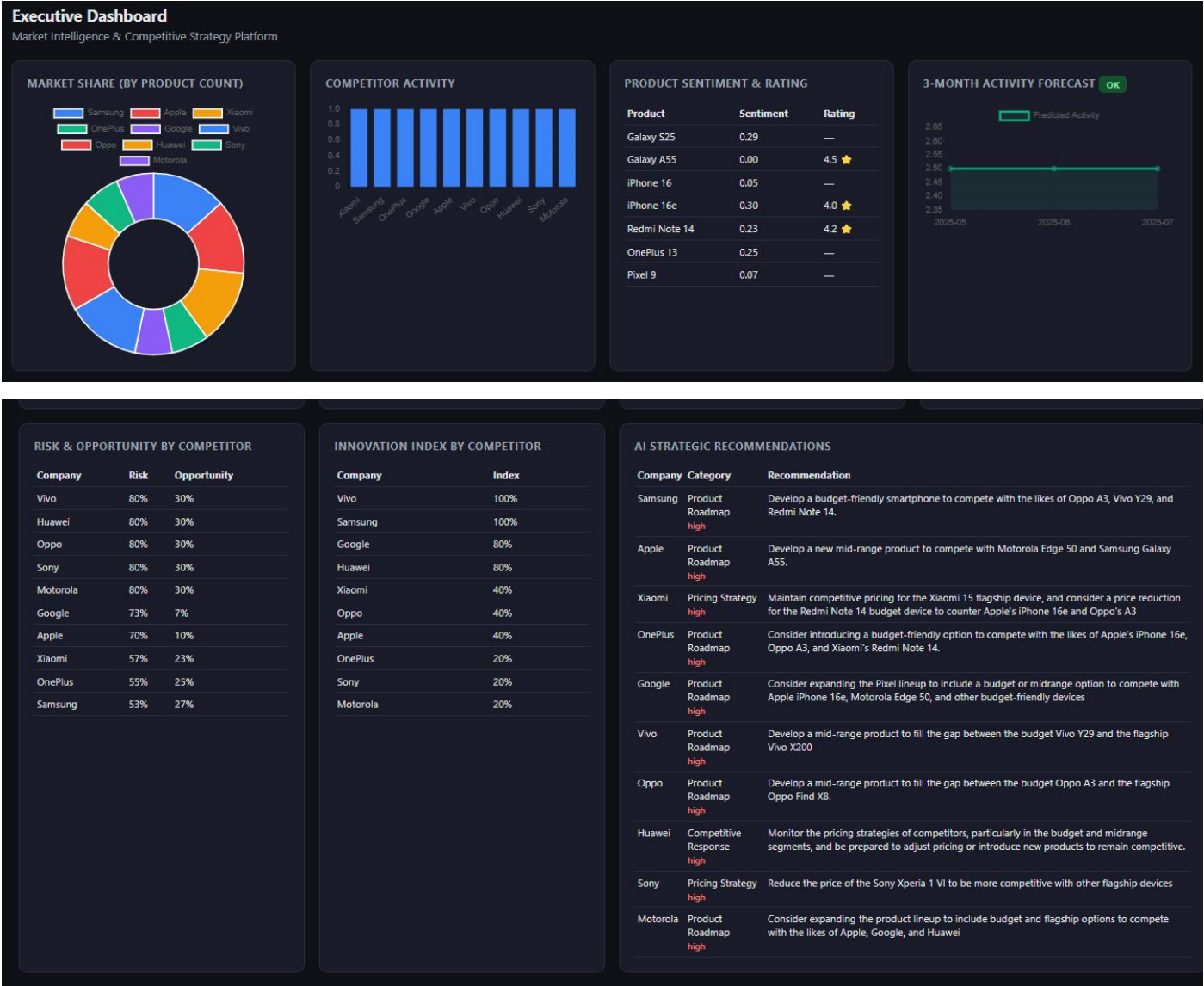


Figure 6: Executive Dashboard

5.14 Multi-Agent AI Architecture

Multi-agent orchestration is implemented through the Executive Copilot's LangGraph state graph, in which the Research and Strategy nodes operate in sequence, sharing a typed state object that is passed through the graph. The remaining reasoning modules (Forecasting, SWOT, Strategy Advisor, Scenario Simulation) each follow the same evidence-then-reasoning pattern independently.

5.15 Redis Caching

Redis, provided by Memurai on Windows, is used to cache four endpoints: the Dashboard, with a 60-second time-to-live given the number of sub-queries it aggregates, and the SWOT, Strategy Advisor, and Forecast endpoints, each with a 10-minute time-to-live, reflecting the cost of the underlying language model calls.

Each cached response includes a `_cache` field indicating whether it was served from cache or computed fresh.

Cache Miss:

```
{
  "market_share_trend": [
    {
      "company": "Samsung",
      "product_count": 2
    },
    {
      "company": "Apple",
      "product_count": 2
    },
    {
      "company": "Xiaomi",
      "product_count": 2
    },
    {
      "company": "OnePlus",
      "product_count": 1
    },
    {
      "company": "Google",
      "product_count": 1
    },
    {
      "company": "Vivo",
      "product_count": 2
    },
    {
      "company": "Oppo",
      "product_count": 2
    },
    {
      "company": "Huawei",
      "product_count": 1
    },
    {
      "company": "Sony",
      "product_count": 1
    },
    {
      "company": "Motorola",
      "product_count": 1
    }
  ],
  "competitor_activity": [
    {
      "company": "Xiaomi",
      "event_count": 1
    },
  ],
}
```



```

{
  "company": "Samsung",
  "event_count": 1
},
{
  "company": "OnePlus",
  "event_count": 1
},
{
  "company": "Google",
  "event_count": 1
},
{
  "company": "Apple",
  "event_count": 1
},
{
  "company": "Vivo",
  "event_count": 1
},
{
  "company": "Oppo",
  "event_count": 1
},
{
  "company": "Huawei",
  "event_count": 1
},
{
  "company": "Sony",
  "event_count": 1
},
{
  "company": "Motorola",
  "event_count": 1
}
],
"sentiment_overview": [
  {
    "product": "Galaxy S25",
    "avg_sentiment": 0.2874615384615385,
    "avg_rating": null
  },
  {
    "product": "Galaxy A55",
    "avg_sentiment": 0,
    "avg_rating": 4.5
  },
  {
    "product": "iPhone 16",
    "avg_sentiment": 0.049416666666666665,
    "avg_rating": null
  },

```

```

{
  "product": "iPhone 16e",
  "avg_sentiment": 0.30133333333333334,
  "avg_rating": 4
},
{
  "product": "Redmi Note 14",
  "avg_sentiment": 0.2288,
  "avg_rating": 4.2
},
{
  "product": "OnePlus 13",
  "avg_sentiment": 0.25266666666666665,
  "avg_rating": null
},
{
  "product": "Pixel 9",
  "avg_sentiment": 0.0745,
  "avg_rating": null
}
],
"overall_forecast": {
  "status": "ok",
  "company": "all_competitors",
  "history_months": 4,
  "forecast": [
    {
      "month": "2025-05",
      "predicted_activity": 2.5,
      "lower_bound": 1.88,
      "upper_bound": 3.11
    },
    {
      "month": "2025-06",
      "predicted_activity": 2.5,
      "lower_bound": 1.83,
      "upper_bound": 3.15
    },
    {
      "month": "2025-07",
      "predicted_activity": 2.5,
      "lower_bound": 1.86,
      "upper_bound": 3.12
    }
  ]
},
"risk_and_opportunity": [
  {
    "company": "Vivo",
    "risk_score": 0.8,
    "opportunity_score": 0.3,
    "note": "Limited review data available"
  }
],

```

```

{
  "company": "Motorola",
  "risk_score": 0.8,
  "opportunity_score": 0.3,
  "note": "Limited review data available"
},
{
  "company": "Huawei",
  "risk_score": 0.8,
  "opportunity_score": 0.3,
  "note": "Limited review data available"
},
{
  "company": "Oppo",
  "risk_score": 0.8,
  "opportunity_score": 0.3,
  "note": "Limited review data available"
},
{
  "company": "Sony",
  "risk_score": 0.8,
  "opportunity_score": 0.3,
  "note": "Limited review data available"
},
{
  "company": "Google",
  "risk_score": 0.73,
  "opportunity_score": 0.07,
  "note": "Based on avg sentiment 0.07 and 1 tracked competitor events"
},
{
  "company": "Apple",
  "risk_score": 0.7,
  "opportunity_score": 0.1,
  "note": "Based on avg sentiment 0.10 and 1 tracked competitor events"
},
{
  "company": "Xiaomi",
  "risk_score": 0.57,
  "opportunity_score": 0.23,
  "note": "Based on avg sentiment 0.23 and 1 tracked competitor events"
},
{
  "company": "OnePlus",
  "risk_score": 0.55,
  "opportunity_score": 0.25,
  "note": "Based on avg sentiment 0.25 and 1 tracked competitor events"
},
{
  "company": "Samsung",
  "risk_score": 0.53,
  "opportunity_score": 0.27,
  "note": "Based on avg sentiment 0.27 and 1 tracked competitor events"
}

```

```

    }
  ],
  "innovation_index": [
    {
      "company": "Vivo",
      "innovation_index": 1,
      "note": "Based on 1 tech/expansion events and 2 tracked products (proxy metric – no patent/R&D data in scope)"
    },
    {
      "company": "Samsung",
      "innovation_index": 1,
      "note": "Based on 1 tech/expansion events and 2 tracked products (proxy metric – no patent/R&D data in scope)"
    },
    {
      "company": "Google",
      "innovation_index": 0.8,
      "note": "Based on 1 tech/expansion events and 1 tracked products (proxy metric – no patent/R&D data in scope)"
    },
    {
      "company": "Huawei",
      "innovation_index": 0.8,
      "note": "Based on 1 tech/expansion events and 1 tracked products (proxy metric – no patent/R&D data in scope)"
    },
    {
      "company": "Xiaomi",
      "innovation_index": 0.4,
      "note": "Based on 0 tech/expansion events and 2 tracked products (proxy metric – no patent/R&D data in scope)"
    },
    {
      "company": "Oppo",
      "innovation_index": 0.4,
      "note": "Based on 0 tech/expansion events and 2 tracked products (proxy metric – no patent/R&D data in scope)"
    },
    {
      "company": "Apple",
      "innovation_index": 0.4,
      "note": "Based on 0 tech/expansion events and 2 tracked products (proxy metric – no patent/R&D data in scope)"
    },
    {
      "company": "Motorola",
      "innovation_index": 0.2,
      "note": "Based on 0 tech/expansion events and 1 tracked products (proxy metric – no patent/R&D data in scope)"
    },
    {
      "company": "Sony",

```

```

    "innovation_index": 0.2,
    "note": "Based on 0 tech/expansion events and 1 tracked products (proxy metric – no
patent/R&D data in scope)"
  },
  {
    "company": "OnePlus",
    "innovation_index": 0.2,
    "note": "Based on 0 tech/expansion events and 1 tracked products (proxy metric – no
patent/R&D data in scope)"
  }
],
"strategic_recommendations": [
  {
    "company": "Samsung",
    "category": "New Market Entry",
    "recommendation": "Launch a budget 5G smartphone ($199-$249) targeting Southeast Asia
and India, where Oppo A3 and Redmi Note 14 have recently cut prices.",
    "priority": "high"
  },
  {
    "company": "Apple",
    "category": "Pricing Strategy",
    "recommendation": "Introduce limited-time discount bundles or trade-in incentives for
the iPhone 16 to narrow the $200-$300 price gap with competing flagships.",
    "priority": "high"
  },
  {
    "company": "Xiaomi",
    "category": "New Market Entry",
    "recommendation": "Accelerate retail and online distribution rollout for the Redmi
Note 14 in Southeast Asia and South Asia, targeting tier-2 and tier-3 cities.",
    "priority": "high"
  },
  {
    "company": "OnePlus",
    "category": "New Market Entry",
    "recommendation": "Launch a focused retail and carrier partnership rollout in
Southeast Asia and South Asia, leveraging the recent OnePlus telecom bundling
partnership.",
    "priority": "high"
  },
  {
    "company": "Google",
    "category": "New Market Entry",
    "recommendation": "Accelerate retail and carrier rollout of Pixel devices in
Southeast Asia and South Asia.",
    "priority": "high"
  },
  {
    "company": "Vivo",
    "category": "New Market Entry",
    "recommendation": "Launch the budget Y29 in Southeast Asia and strengthen
distribution of the X200 flagship in the same region.",

```

```

    "priority": "high"
  },
  {
    "company": "Oppo",
    "category": "Pricing Strategy",
    "recommendation": "Introduce a limited-time promotional discount of $50-$70 on the Find X8 flagship in Southeast Asia and launch bundled accessories (e.g., earbuds, case) to increase perceived value without eroding the premium price point.",
    "priority": "high"
  },
  {
    "company": "Huawei",
    "category": "Pricing Strategy",
    "recommendation": "Introduce limited-time promotional discounts for the Pura 70 in Southeast Asia to counter recent price cuts by Oppo, Xiaomi, and Motorola.",
    "priority": "high"
  },
  {
    "company": "Sony",
    "category": "Pricing Strategy",
    "recommendation": "Introduce a tiered pricing model for the Xperia line, including a lower-priced flagship variant or a premium-midrange model priced around $899-$949 to align with competitor flagship pricing.",
    "priority": "high"
  },
  {
    "company": "Motorola",
    "category": "Pricing Strategy",
    "recommendation": "Introduce a limited-time promotional bundle for the Edge 50 (e.g., free accessories or a 6-month data plan) in Southeast Asia to offset the recent 12% price cut and keep the effective price competitive with Samsung Galaxy A55 ($449) and new budget offerings.",
    "priority": "high"
  }
],
"_cache": "miss"
}

```

Cache Hit:

```

{
  "market_share_trend": [
    {
      "company": "Samsung",
      "product_count": 2
    },
    {
      "company": "Apple",
      "product_count": 2
    },
    {
      "company": "Xiaomi",
      "product_count": 2
    }
  ]
}

```

```

    },
    {
      "company": "OnePlus",
      "product_count": 1
    },
    {
      "company": "Google",
      "product_count": 1
    },
    {
      "company": "Vivo",
      "product_count": 2
    },
    {
      "company": "Oppo",
      "product_count": 2
    },
    {
      "company": "Huawei",
      "product_count": 1
    },
    {
      "company": "Sony",
      "product_count": 1
    },
    {
      "company": "Motorola",
      "product_count": 1
    }
  ],
  "competitor_activity": [
    {
      "company": "Xiaomi",
      "event_count": 1
    },
    {
      "company": "Samsung",
      "event_count": 1
    },
    {
      "company": "OnePlus",
      "event_count": 1
    },
    {
      "company": "Google",
      "event_count": 1
    },
    {
      "company": "Apple",
      "event_count": 1
    },
    {
      "company": "Vivo",

```

```

    "event_count": 1
  },
  {
    "company": "Oppo",
    "event_count": 1
  },
  {
    "company": "Huawei",
    "event_count": 1
  },
  {
    "company": "Sony",
    "event_count": 1
  },
  {
    "company": "Motorola",
    "event_count": 1
  }
],
"sentiment_overview": [
  {
    "product": "Galaxy S25",
    "avg_sentiment": 0.2874615384615385,
    "avg_rating": null
  },
  {
    "product": "Galaxy A55",
    "avg_sentiment": 0,
    "avg_rating": 4.5
  },
  {
    "product": "iPhone 16",
    "avg_sentiment": 0.049416666666666665,
    "avg_rating": null
  },
  {
    "product": "iPhone 16e",
    "avg_sentiment": 0.30133333333333334,
    "avg_rating": 4
  },
  {
    "product": "Redmi Note 14",
    "avg_sentiment": 0.2288,
    "avg_rating": 4.2
  },
  {
    "product": "OnePlus 13",
    "avg_sentiment": 0.25266666666666665,
    "avg_rating": null
  },
  {
    "product": "Pixel 9",
    "avg_sentiment": 0.0745,

```



```

    "avg_rating": null
  }
],
"overall_forecast": {
  "status": "ok",
  "company": "all_competitors",
  "history_months": 4,
  "forecast": [
    {
      "month": "2025-05",
      "predicted_activity": 2.5,
      "lower_bound": 1.88,
      "upper_bound": 3.11
    },
    {
      "month": "2025-06",
      "predicted_activity": 2.5,
      "lower_bound": 1.83,
      "upper_bound": 3.15
    },
    {
      "month": "2025-07",
      "predicted_activity": 2.5,
      "lower_bound": 1.86,
      "upper_bound": 3.12
    }
  ]
},
"risk_and_opportunity": [
  {
    "company": "Vivo",
    "risk_score": 0.8,
    "opportunity_score": 0.3,
    "note": "Limited review data available"
  },
  {
    "company": "Motorola",
    "risk_score": 0.8,
    "opportunity_score": 0.3,
    "note": "Limited review data available"
  },
  {
    "company": "Huawei",
    "risk_score": 0.8,
    "opportunity_score": 0.3,
    "note": "Limited review data available"
  },
  {
    "company": "Oppo",
    "risk_score": 0.8,
    "opportunity_score": 0.3,
    "note": "Limited review data available"
  }
],

```

```

{
  "company": "Sony",
  "risk_score": 0.8,
  "opportunity_score": 0.3,
  "note": "Limited review data available"
},
{
  "company": "Google",
  "risk_score": 0.73,
  "opportunity_score": 0.07,
  "note": "Based on avg sentiment 0.07 and 1 tracked competitor events"
},
{
  "company": "Apple",
  "risk_score": 0.7,
  "opportunity_score": 0.1,
  "note": "Based on avg sentiment 0.10 and 1 tracked competitor events"
},
{
  "company": "Xiaomi",
  "risk_score": 0.57,
  "opportunity_score": 0.23,
  "note": "Based on avg sentiment 0.23 and 1 tracked competitor events"
},
{
  "company": "OnePlus",
  "risk_score": 0.55,
  "opportunity_score": 0.25,
  "note": "Based on avg sentiment 0.25 and 1 tracked competitor events"
},
{
  "company": "Samsung",
  "risk_score": 0.53,
  "opportunity_score": 0.27,
  "note": "Based on avg sentiment 0.27 and 1 tracked competitor events"
}
],
"innovation_index": [
  {
    "company": "Vivo",
    "innovation_index": 1,
    "note": "Based on 1 tech/expansion events and 2 tracked products (proxy metric – no patent/R&D data in scope)"
  },
  {
    "company": "Samsung",
    "innovation_index": 1,
    "note": "Based on 1 tech/expansion events and 2 tracked products (proxy metric – no patent/R&D data in scope)"
  },
  {
    "company": "Google",
    "innovation_index": 0.8,

```

```

    "note": "Based on 1 tech/expansion events and 1 tracked products (proxy metric – no
patent/R&D data in scope)"
  },
  {
    "company": "Huawei",
    "innovation_index": 0.8,
    "note": "Based on 1 tech/expansion events and 1 tracked products (proxy metric – no
patent/R&D data in scope)"
  },
  {
    "company": "Xiaomi",
    "innovation_index": 0.4,
    "note": "Based on 0 tech/expansion events and 2 tracked products (proxy metric – no
patent/R&D data in scope)"
  },
  {
    "company": "Oppo",
    "innovation_index": 0.4,
    "note": "Based on 0 tech/expansion events and 2 tracked products (proxy metric – no
patent/R&D data in scope)"
  },
  {
    "company": "Apple",
    "innovation_index": 0.4,
    "note": "Based on 0 tech/expansion events and 2 tracked products (proxy metric – no
patent/R&D data in scope)"
  },
  {
    "company": "Motorola",
    "innovation_index": 0.2,
    "note": "Based on 0 tech/expansion events and 1 tracked products (proxy metric – no
patent/R&D data in scope)"
  },
  {
    "company": "Sony",
    "innovation_index": 0.2,
    "note": "Based on 0 tech/expansion events and 1 tracked products (proxy metric – no
patent/R&D data in scope)"
  },
  {
    "company": "OnePlus",
    "innovation_index": 0.2,
    "note": "Based on 0 tech/expansion events and 1 tracked products (proxy metric – no
patent/R&D data in scope)"
  }
],
"strategic_recommendations": [
  {
    "company": "Samsung",
    "category": "New Market Entry",
    "recommendation": "Launch a budget 5G smartphone ($199-$249) targeting Southeast Asia
and India, where Oppo A3 and Redmi Note 14 have recently cut prices.",
    "priority": "high"
  }
]

```

```

    },
    {
      "company": "Apple",
      "category": "Pricing Strategy",
      "recommendation": "Introduce limited-time discount bundles or trade-in incentives for the iPhone 16 to narrow the $200-$300 price gap with competing flagships.",
      "priority": "high"
    },
    {
      "company": "Xiaomi",
      "category": "New Market Entry",
      "recommendation": "Accelerate retail and online distribution rollout for the Redmi Note 14 in Southeast Asia and South Asia, targeting tier-2 and tier-3 cities.",
      "priority": "high"
    },
    {
      "company": "OnePlus",
      "category": "New Market Entry",
      "recommendation": "Launch a focused retail and carrier partnership rollout in Southeast Asia and South Asia, leveraging the recent OnePlus telecom bundling partnership.",
      "priority": "high"
    },
    {
      "company": "Google",
      "category": "New Market Entry",
      "recommendation": "Accelerate retail and carrier rollout of Pixel devices in Southeast Asia and South Asia.",
      "priority": "high"
    },
    {
      "company": "Vivo",
      "category": "New Market Entry",
      "recommendation": "Launch the budget Y29 in Southeast Asia and strengthen distribution of the X200 flagship in the same region.",
      "priority": "high"
    },
    {
      "company": "Oppo",
      "category": "Pricing Strategy",
      "recommendation": "Introduce a limited-time promotional discount of $50-$70 on the Find X8 flagship in Southeast Asia and launch bundled accessories (e.g., earbuds, case) to increase perceived value without eroding the premium price point.",
      "priority": "high"
    },
    {
      "company": "Huawei",
      "category": "Pricing Strategy",
      "recommendation": "Introduce limited-time promotional discounts for the Pura 70 in Southeast Asia to counter recent price cuts by Oppo, Xiaomi, and Motorola.",
      "priority": "high"
    },
    {

```

```

    "company": "Sony",
    "category": "Pricing Strategy",
    "recommendation": "Introduce a tiered pricing model for the Xperia line, including a
lower-priced flagship variant or a premium-midrange model priced around $899-$949 to align
with competitor flagship pricing.",
    "priority": "high"
  },
  {
    "company": "Motorola",
    "category": "Pricing Strategy",
    "recommendation": "Introduce a limited-time promotional bundle for the Edge 50 (e.g.,
free accessories or a 6-month data plan) in Southeast Asia to offset the recent 12% price
cut and keep the effective price competitive with Samsung Galaxy A55 ($449) and new budget
offerings.",
    "priority": "high"
  }
],
"_cache": "hit"
}

```

5.16 NetworkX Competitive Analysis

This module builds a graph in which companies are connected when their products share a product category and price tier, with edge weight reflecting how many attributes are shared. Weighted degree centrality is computed and normalized to produce a ranking of which competitors face the broadest overlapping competition. An earlier version using category alone produced an uninformative result in which every company scored identically; adding price tier as a second shared attribute resolved this.

```

{
  "status": "ok",
  "method": "weighted degree centrality over a graph where companies are linked by shared
product category and/or price tier (normalized against the most-contested company)",
  "rankings": [
    {
      "company": "Apple",
      "centrality_score": 1,
      "raw_shared_links": 21
    },
    {
      "company": "Xiaomi",
      "centrality_score": 0.952,
      "raw_shared_links": 20
    },
    {
      "company": "Vivo",
      "centrality_score": 0.952,
      "raw_shared_links": 20
    },
    {
      "company": "Oppo",

```

```

    "centrality_score": 0.952,
    "raw_shared_links": 20
  },
  {
    "company": "Samsung",
    "centrality_score": 0.905,
    "raw_shared_links": 19
  },
  {
    "company": "OnePlus",
    "centrality_score": 0.714,
    "raw_shared_links": 15
  },
  {
    "company": "Huawei",
    "centrality_score": 0.714,
    "raw_shared_links": 15
  },
  {
    "company": "Sony",
    "centrality_score": 0.714,
    "raw_shared_links": 15
  },
  {
    "company": "Google",
    "centrality_score": 0.524,
    "raw_shared_links": 11
  },
  {
    "company": "Motorola",
    "centrality_score": 0.19,
    "raw_shared_links": 4
  }
]
}

```

6. REST API Reference

6.1 API Overview

The platform exposes its functionality through a REST API built with FastAPI. All endpoints are documented and testable through the interactive documentation page at <http://127.0.0.1:8000/docs> once the backend server is running.

6.2 Endpoint Summary

Here is the table from the image formatted in Markdown:

Table 3: Endpoint Summary

Method	Endpoint	Description
GET	/api/competitors	Lists all tracked competitor companies
GET	/api/forecast/trend	Returns a three-month activity forecast across all competitors
GET	/api/forecast/trend/{competitor_name}	Returns a three-month forecast for a specific competitor
GET	/api/swot/{company_name}	Returns a SWOT analysis for a company
GET	/api/strategy-advisor/{company_name}	Returns strategic recommendations for a company
POST	/api/scenario	Simulates a hypothetical market scenario
POST	/api/strategy/ask	Submits a question to the AI Executive Copilot
GET	/api/dashboard	Returns aggregated Executive Dashboard data
GET	/api/network-analysis/centrality	Returns competitive centrality rankings

6.3 Request and Response Examples

7. Database Design

7.1 PostgreSQL Schema

raw_articles

Table 4: raw_articles

Column	Type
id	SERIAL PRIMARY KEY

Column	Type
source	TEXT
url	TEXT
title	TEXT
content	TEXT
published_date	DATE
company_mentioned	TEXT
processed	BOOLEAN DEFAULT FALSE
created_at	TIMESTAMP DEFAULT NOW()

raw_reviews

Table 5: raw_reviews

Column	Type
id	SERIAL PRIMARY KEY
product_name	TEXT
source	TEXT
review_text	TEXT
rating	FLOAT, nullable
review_date	DATE
processed	BOOLEAN DEFAULT FALSE
created_at	TIMESTAMP DEFAULT NOW()

7.2 Neo4j Knowledge Graph Schema

Node labels and properties

Table 6: Node labels and Properties

Label	Properties
Company	name, type, headquarters
Product	name, category, price, launch_date
Event	id, type, date, description
Region	name
Article	id, title
Review	id, text, rating, sentiment

Relationship types

Table 6: Relationship Types

Relationship	Direction	Meaning
LAUNCHED	Company to Product	A company brought a product to market
INVOLVED_IN	Company to Event	A company is party to a tracked market event
MENTIONS	Article to Company	An article discusses a company
REVIEWS	Review to Product	A review pertains to a specific product

7.3 ChromaDB Collection Structure

A single persistent collection, `market_intel_docs`, stores embedded vectors for both articles and reviews. Each item's metadata includes a `type` field (article or review) along with source-specific fields: `company` and `date` for articles, and `product`, `date`, and `rating` for reviews.

8. Deployment Overview

8.1 System Requirements

The system requires a Windows 10 or Windows 11 environment with Python 3.11 or a compatible Python 3.x version. A minimum of 8 GB RAM is recommended, while 16 GB or more provides better performance when running multiple services simultaneously. At least 10 GB of free disk space is recommended for the application, dependencies, cached data, and generated files.

A stable internet connection is required because the platform collects live data from external sources and uses cloud-based AI services. PostgreSQL, Neo4j, and Redis (Memurai) must also be installed and running before starting the complete system.

8.2 Software Components

The main software components required for deployment are:

- Python 3.11 or compatible Python 3.x version
- FastAPI and Uvicorn for the backend API
- PostgreSQL for raw data storage
- Neo4j for the Knowledge Graph
- Redis (Memurai) for caching
- ChromaDB and Sentence Transformers for semantic search and RAG
- Prophet for time-series forecasting
- spaCy and TextBlob for NLP processing
- NetworkX for competitive network analysis
- Groq API for LLM-based AI features
- A modern web browser for accessing the Executive Dashboard

8.3 Deployment Workflow

The deployment process follows a sequential workflow. First, the required software components are installed and the Python environment is configured. The required Python dependencies are then installed using the project's **requirements.txt** file.

Next, PostgreSQL, Neo4j, and Redis (Memurai) are configured and started. The required environment variables and API credentials are added to the **.env** configuration file. After the databases are ready, the initial database and Knowledge Graph setup scripts are executed.

The FastAPI backend is then started using Uvicorn. Once the backend is running, the data ingestion scripts can collect and process market information from the supported sources. The processed information is stored in PostgreSQL and Neo4j, while the RAG index and other AI components use the processed data for analysis.

Finally, the Executive Dashboard is opened in a web browser. Automated data updates are configured through Windows Task Scheduler to refresh the platform every six hours without requiring manual execution.

8.4 Environment Configuration Overview

The platform uses an environment configuration file to store database connection details and external API credentials. The `.env` file contains configuration values such as PostgreSQL credentials, Neo4j connection information, Redis configuration, and API keys required by external services.

Actual credentials and API keys should not be included in the source code or public GitHub repository. A `.env.example` file can be provided as a template so that a new deployment can be configured safely.

Detailed installation commands, database configuration, environment variable setup, API configuration, Task Scheduler setup, troubleshooting procedures, and other deployment steps are provided in the separate **Deployment & Installation Guide**.

9. Testing and Validation

9.1 Functional Testing

Each module was tested individually after implementation and again after every subsequent change that touched shared components. Testing consisted of running the relevant script or API endpoint, inspecting the raw output, and confirming it matched expected behavior rather than assuming correctness from the absence of an error.

9.2 API Testing

All nine REST endpoints were tested through FastAPI's interactive documentation interface at `/docs`, using representative inputs for each tracked company. Testing covered both the "happy path" (a company with sufficient data, producing a confident, evidence-backed response) and the sparse-data path (a company with limited events or reviews, producing an explicitly low-confidence or **insufficient_data** response).

9.3 Knowledge Graph Verification

The Neo4j knowledge graph was verified directly using queries in Neo4j at each stage of development:

- Node and relationship counts were checked after every data load to confirm the expected number of Company, Product, and Event nodes existed
- MENTIONS and REVIEWS relationships were checked after each NLP processing run to confirm articles and reviews were being linked to the correct Company and Product nodes

9.4 Semantic Search Validation

The ChromaDB-based semantic search index was validated with a test query, "Which competitor changed pricing recently?", chosen because it shares no exact words with the underlying source articles. The query correctly returned the Xiaomi Redmi Note 14 price-cut article as the top result, followed by the Apple iPhone 16e budget-launch article and a related product review, confirming that retrieval was working by meaning rather than by keyword overlap.

9.5 Forecasting Validation

The forecasting engine was tested against both sufficient and insufficient data conditions. With limited per-company event history, the module correctly returned an **insufficient_data** status rather than producing an unreliable forecast. With the aggregate, cross-company event history, the module produced a three-month forecast with declining predicted activity and plausible upper and lower confidence bounds, confirming the model was fitting correctly rather than returning placeholder values.

9.6 Dashboard Validation

The Executive Dashboard was validated by confirming that all seven display cards, Market Share, Competitor Activity, Product Sentiment, the three-month Forecast, Risk and Opportunity, Innovation Index, and AI Strategic Recommendations, rendered with real, non-empty data pulled live from the backend aggregator endpoint.

10. Engineering Challenges and Solutions

10.1 Duplicate Knowledge Graph Nodes

Issue: Rerunning the event-loading script produced ten Event nodes in the knowledge graph instead of the expected five.

Diagnosis: Direct inspection in Neo4j Browser confirmed each of the five intended events existed twice.

Cause: The Cypher query used CREATE to insert Event nodes. CREATE always inserts a new node regardless of whether an equivalent node already exists, so rerunning the script duplicated every event.

Solution: The query was rewritten to use MERGE (ev:Event {id: \$id}) keyed on a stable per-event identifier, followed by a SET clause for the remaining properties. This made the load script idempotent, safe to rerun any number of times without creating duplicates.

10.2 Prophet Build Issues on Windows

Issue: Attempting to use the Prophet forecasting library failed with **AttributeError: 'Prophet' object has no attribute 'stan_backend'**.

Diagnosis: This error indicates Prophet's underlying Stan model failed to compile. Investigation showed the required build command, **mingw32-make**, was resolving to an older, pre-existing MinGW installation elsewhere on the system PATH instead of the one bundled with the RTools toolchain that Prophet's own installer had set up.

Solution: The RTools C++ toolchain was installed via **cmdstanpy install_cxx_toolchain** helper, and the system PATH was corrected so that both the toolchain's **usr/bin** and **mingw64/bin** folders were found ahead of the conflicting installation. After this correction, the Stan backend compiled successfully and Prophet functioned as expected.

10.3 Reddit API Limitations

Issue: An initial approach to sourcing live customer reviews used Reddit's public search endpoint. All requests returned HTTP 403 Blocked.

Diagnosis: Reddit had tightened its public API access policy, requiring authenticated (OAuth) access for automated requests of this kind.

Solution: Rather than implement full OAuth registration for a data source of secondary importance, the review-fetching approach was redesigned to reuse the already-functioning NewsAPI integration, searching with a "review" query term instead. This kept the review pipeline consistent with the existing article-fetching pattern and avoided the added complexity of a second authentication scheme.

10.4 JSON Parsing Issues

Issue: The Scenario Simulation endpoint returned a **parse_error** status despite the underlying language model producing a well-formed, well-reasoned analysis.

Diagnosis: Inspection of the raw model response showed it was wrapped in a markdown code fence (three backticks, optionally followed by "json"), which is not valid input to a standard JSON parser.

Solution: A preprocessing step was added to strip a leading and trailing code fence, and an optional language tag, before parsing. This fix was subsequently applied as a shared safeguard in the Strategy Advisor service as well, since both modules rely on the same language model to return structured output.

10.5 Executive Copilot Evidence Bug

Issue: The AI Executive Copilot returned identical evidence and a confidence score of zero for two unrelated questions, one about competitor growth and one about pricing strategy.

Diagnosis: Tracing the code showed the Research agent's function accepted the user's question as a parameter but never used it, always executing the same two fixed Cypher queries regardless of what was asked.

Solution: Keyword-based routing was added to the Research agent, inspecting the question text for terms related to pricing, market events, sentiment, or regional presence, and selecting the corresponding graph queries accordingly, with a broad fallback when no keywords matched. After the fix, the same pricing question correctly retrieved live product pricing data and returned a grounded answer with 80 percent confidence.

10.6 NewsAPI Rate Limiting

Issue: After expanding tracked competitors from five to ten companies, the live update cycle began failing with HTTP 429 Too Many Requests.

Diagnosis: Four fetch scripts, each issuing one request per tracked company or product, resulted in approximately 45 API calls per cycle. At the original 30-minute Task Scheduler interval, this projected to well over 2,000 calls per day against NewsAPI's free-tier limit of 100 requests per day.

Solution: The Task Scheduler interval was changed from 30 minutes to 6 hours, bringing daily call volume within the free-tier quota while still meeting the case study's near-real-time updates requirement at this data scale.

10.7 NetworkX Centrality Improvements

Issue: The competitive centrality analysis initially returned an identical score of 1.0 for every tracked company.

Diagnosis: The graph connected companies whenever they shared a single product category. At the platform's data scale, with each company having at least one flagship-category product, this produced a fully connected graph in which every node had the same degree, making the centrality score uninformative despite being mathematically correct.

Solution: A second shared attribute, price tier, was added alongside product category, and the calculation was changed from unweighted to weighted degree centrality. This produced meaningfully differentiated rankings, even at the platform's current data scale.

11. Known Limitations and Future Work

11.1 Current Limitations

Smaller data size. The platform tracks 10 competitor companies and 1 product type (smartphones), instead of the case study's request for 250 competitors across 18 product types and 60 countries. This choice was made so the full system could be built and tested properly in 4 weeks, instead of building only part of a bigger system.

Three data sources not built. Out of the 9 data source types listed in the case study, 6 are working and live: News Articles, Industry Publications, Public Report Coverage, Customer Reviews, Financial Reports, and Patent Databases. Company Websites, Product Catalogs, and Social Media were not built. Every company's website looks different and has no shared way to pull data from it, which would mean building and maintaining a separate tool for each company. Social media needs either a paid API (Twitter/X) or special login access that wasn't set up in time (Reddit blocked requests without it).

Financial Reports only cover 2 companies. Out of the 10 tracked companies, only Apple and Google are registered with the US SEC, so only they show up in this data source. The other 8 companies are not included here.

Market events are added by hand, not found automatically. Things like price changes and partnerships are currently entered manually into the knowledge graph, not automatically detected from live news articles. The system currently detects company names and review sentiment from text, but not event types yet.

The Innovation Index is an estimate, not real patent data. Since there wasn't enough real patent or R&D spending data to use, the Innovation Index shown on the dashboard is calculated using two things that were available: how many technology-related events a company has, and how many products it has. It is not based on actual patent counts.

11.2 Future Enhancements

- Automatically detect event types (price change, new launch, partnership, etc.) from live articles, instead of entering them by hand.
- Add financial report coverage for companies not registered with the US SEC, using their country's equivalent filing system.
- Improve the Innovation Index using real patent counts, once enough patent data has been collected over time.
- Add website or product catalog data for a few important companies first, instead of all 10 at once, to make it easier to maintain.
- Add social media data once there is time to set up proper login/authentication access.
- Track more companies and more product categories, the current system is built to support this without needing to be redesigned.
- Move the platform to cloud hosting, so it doesn't depend on one local computer and its task scheduler.

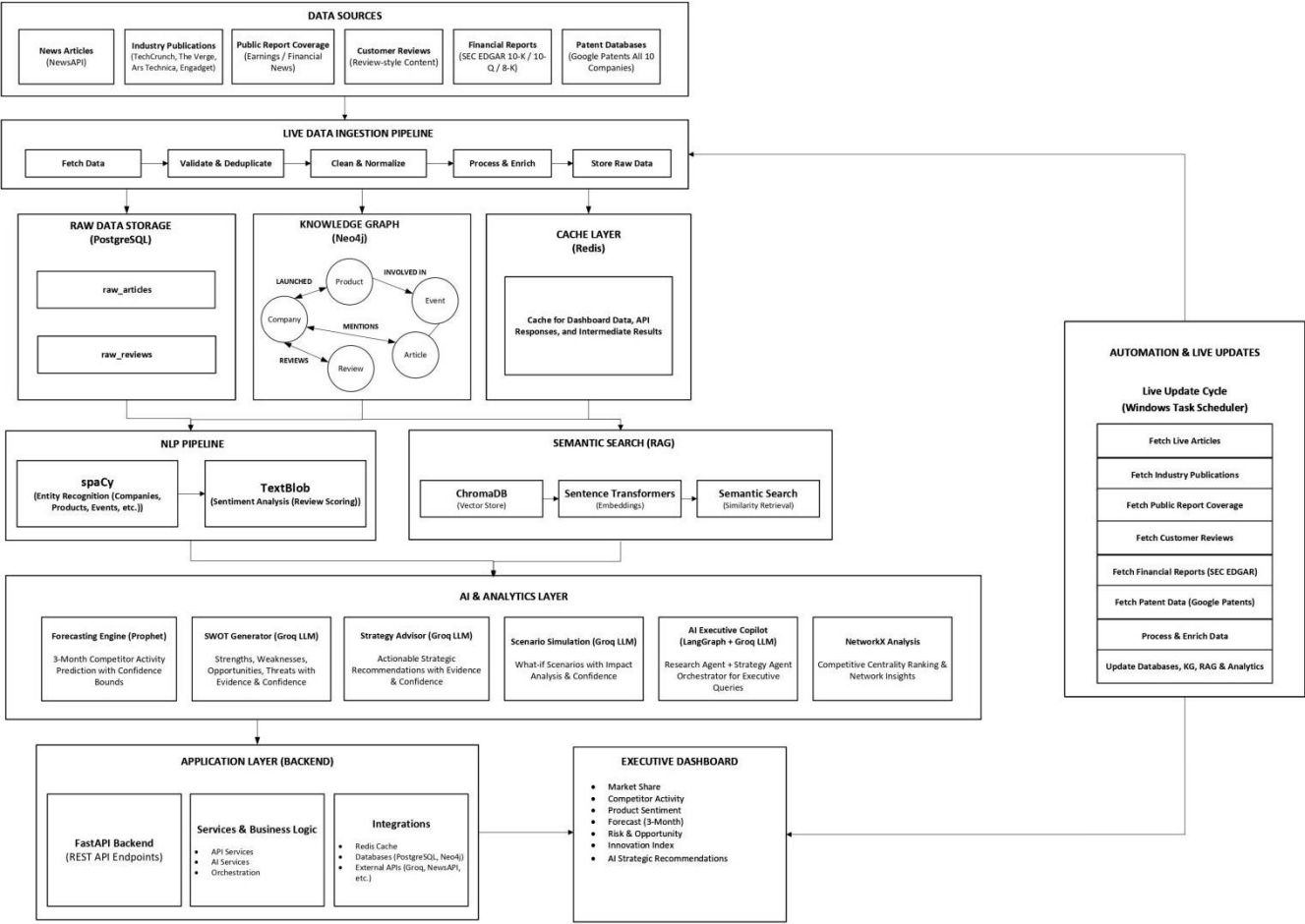
12. Conclusion

This project built the complete system described in the AI-004 case study an Enterprise Autonomous Market Intelligence & Competitive Strategy Platform in 4 weeks. Instead of cutting down individual features, the approach was to build every required part in full, and instead reduce the amount of data used.

During development, the project grew bigger than originally planned. It started with sample data entered by hand, and became a system that automatically pulls real data from 6 live internet sources on a repeating schedule, covering 10 real competitor companies. Real problems were found through actual testing, not assumed to be fine, including a data duplication issue in the knowledge graph, a Windows-specific setup problem in the forecasting tool, a blocked API from a third party, a response-reading error, and a logic mistake in the AI assistant's evidence search. Each one is explained in this report, including what caused it and how it was fixed.

The result is a platform where every AI-generated result whether a SWOT analysis, a strategy recommendation, a scenario prediction, or an assistant's answer can always be traced back to real evidence, and always comes with an honest confidence level, including cases where the system says it doesn't have enough data to be sure. This was treated as a core requirement throughout the project, directly matching the case study's Explainable AI requirement.

Appendix A. Project Architecture Diagram



Appendix B. API Endpoints Summary

Method	Endpoint	Description	Response Includes
GET	/api/competitors	Lists all tracked competitor companies	Company names
GET	/api/forecast/trend	Three-month activity forecast across all competitors	status, forecast values, confidence bounds
GET	/api/forecast/trend/{competitor_name}	Three-month activity forecast for a specific competitor	status, forecast values, confidence bounds

Method	Endpoint	Description	Response Includes
GET	/api/swot/{company_name}	SWOT analysis for a company	strengths, weaknesses, opportunities, threats, evidence, confidence
GET	/api/strategy-advisor/{company_name}	Strategic recommendations for a company	recommendations by category, evidence, priority, confidence
POST	/api/scenario	Simulates a hypothetical market scenario	predicted impacts, severity, recommended response, confidence
POST	/api/strategy/ask	AI Executive Copilot question and answer	answer, evidence used, confidence
GET	/api/dashboard	Aggregated Executive Dashboard data	market share, competitor activity, sentiment, forecast, risk and opportunity, innovation index, strategic recommendations
GET	/api/network-analysis/centrality	Competitive centrality ranking	ranked company list, centrality scores

Appendix C. Environment Variables

The following variables are required in **backend/.env**:

Variable	Purpose
NEO4J_URI	Connection URI for the Neo4j database (default: neo4j://127.0.0.1:7687)
NEO4J_USER	Neo4j username

Variable	Purpose
NEO4J_PASSWORD	Neo4j password
POSTGRES_HOST	PostgreSQL host (default: localhost)
POSTGRES_PORT	PostgreSQL port (default: 5432)
POSTGRES_DB	PostgreSQL database name (marketintel_raw)
POSTGRES_USER	PostgreSQL username
POSTGRES_PASSWORD	PostgreSQL password
GROQ_API_KEY	API key for the Groq LLM, used by SWOT, Strategy Advisor, Scenario Simulation, and the Executive Copilot
NEWSAPI_KEY	API key for NewsAPI, used by all live article, publication, report, and review fetch scripts